Universidad Autónoma de Madrid

Departamento de Informática

Sistemas Informáticos II
Práctica - 4

David Teófilo Garitagoitia Romero
Roberto Marabini

Registro de Cambios

Versión[1]	**Fecha**	**Autor**	**Descripción**
1.0	28.3.2025	DT	Primera versión.
2.0	30.9.2025	RM	Revisión curso 2025-26.
2.1	15.1.2026	RM	Revisión previa al comienzo del curso

[1]La asignación de versiones se realizan mediante 2 números $X.Y$. Cambios en Y indican aclaraciones o descripciones más detalladas de algún punto. Cambios en X indican modificaciones más profundas que o bien varían el material suministrado o el contenido de la práctica.

Contents

1 Objetivos **3**

2 Descripción del Proyecto **5**

2.1 Objetivos específicos 5

2.2 Componentes 5

2.3 Flujo de datos 6

3 Instrucciones para la configuración y el despliegue **8**

3.1 Configuración de Apache para el Balanceo de Carga 8

3.1.1 Configuración del Proxy Balancer 8

3.1.2 Explicación de las Directivas 10

3.1.3 Verificación de la Configuración 11

4 Balanceo de carga **13**

4.1 Afinidad de sesión 15

5 Pruebas balanceador **16**

6 Criterios de evaluación **21**

1 Objetivos

En el ámbito de la informática, **la escalabilidad** y **la disponibilidad** son principios esenciales en el diseño y despliegue de soluciones software. Mientras la escalabilidad permite que una aplicación se adapte al aumento de la demanda sin degradar su rendimiento, la disponibilidad garantiza la continuidad del servicio ante fallos parciales de la infraestructura.

- **Escalabilidad:** Capacidad de una aplicación para ajustarse y crecer según las demandas de carga. Puede lograrse de dos formas:
  - **Escalabilidad vertical:** aumento de recursos (CPU, RAM, almacenamiento, GPU, etc.) en una misma máquina para mejorar su capacidad de procesamiento.
  - **Escalabilidad horizontal:** adición de nuevas instancias o servidores que trabajen en conjunto para distribuir la carga y aumentar la capacidad global del sistema.
- **Disponibilidad:** Diseño de sistemas que continúan funcionando de manera correcta incluso cuando uno o varios componentes fallan. Su propósito principal es minimizar los períodos de inactividad, garantizando que el servicio esté disponible en todo momento.

El objetivo de esta práctica es implementar una solución que combine **disponibilidad** y **escalabilidad** en una aplicación Django, empleando un clúster de servidores, un balanceador de carga y la técnica de *sticky sessions*.

A continuación se describen algunos términos que serán usados en esta guía.

- **Clúster:** Conjunto de servidores interconectados que reparten la carga de trabajo y ofrecen redundancia. Un clúster permite escalar horizontalmente y mantener el servicio operativo si uno de los nodos falla.
- **Balanceador de carga:** Componente que distribuye las solicitudes entre instancias para evitar la sobrecarga de un único servidor. También contribuye a la alta disponibilidad, redirigiendo tráfico en caso de fallo. Métodos comunes:

  - *Round-robin:* repartir solicitudes de forma secuencial entre servidores.
  - *Least connections:* dirigir solicitudes al servidor con menor número de conexiones activas.

- **Sticky sessions:** Técnica que asegura que todas las solicitudes de un mismo usuario se asignen al mismo servidor durante la sesión. Es importante cuando la sesión se almacena localmente en el servidor y no en un recurso compartido como una base de datos centralizada.

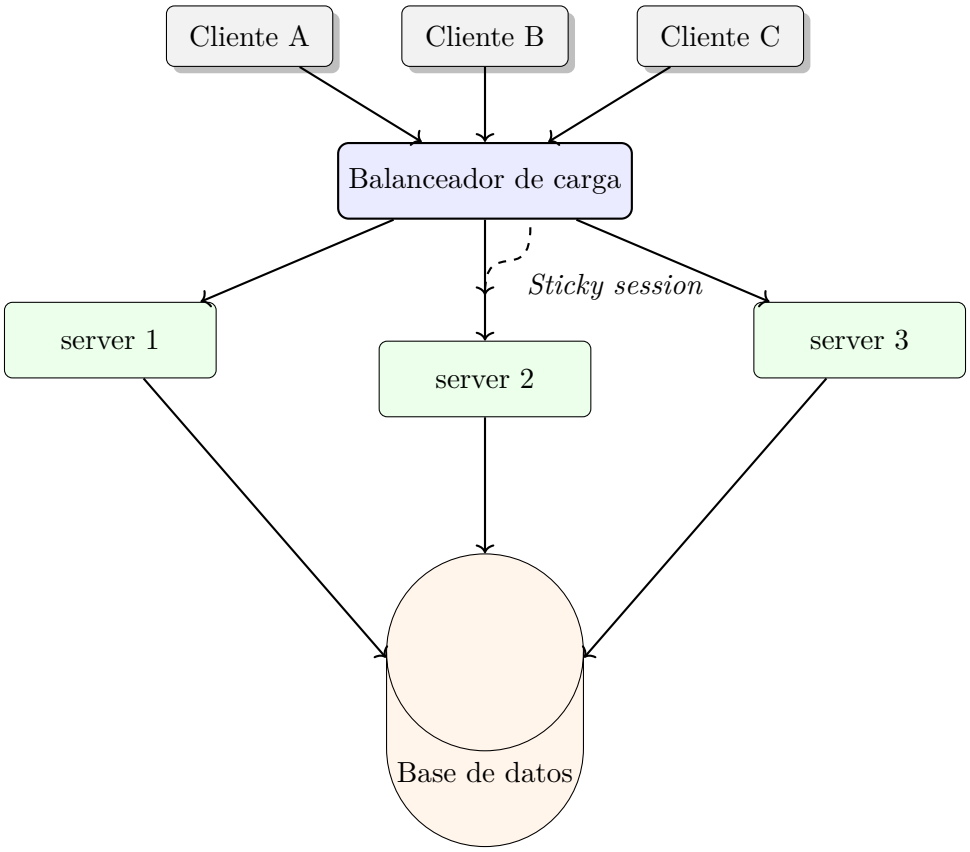

Figure 1: Arquitectura con balanceador, clúster de servidores y almacenamiento compartido. Si la "sticky session" está habilitada todas las peticiones de un mismo cliente se redirigen al mismo servidor.

2 Descripción del Proyecto

Una propiedad fundamental que deben satisfacer los sistemas distribuidos es la **transparencia**. Este concepto se refiere a la capacidad del sistema para ocultar su estructura interna al usuario (ya sea final o desarrollador), proporcionando una interfaz homogénea independientemente de las variaciones subyacentes.

Gracias a esta transparencia, los sistemas distribuidos pueden operar de manera eficiente sin importar la carga de trabajo (transparencia de escalado) ni las posibles incidencias que puedan provocar fallos en la operación (transparencia frente a fallos).

La estrategia más habitual para cumplir con estos requisitos consiste en ejecutar el servicio en múltiples nodos de procesamiento. Estos conjuntos de servidores que ofrecen servicios equivalentes se conocen como *clusters*.

En esta práctica se parte de la aplicación desarrollada en ejercicios anteriores: un sistema de pago electrónico que permite crear, eliminar y listar transacciones realizadas con tarjetas de crédito.

El objetivo del presente proyecto es configurar y desplegar dicha aplicación *Django* en un entorno de clúster, utilizando Apache como balanceador de carga para distribuir las solicitudes entre varias instancias de la aplicación.

2.1 Objetivos específicos

Los objetivos específicos del sistema son los siguientes:

- Desplegar una aplicación *Django* en un entorno distribuido y escalable.
- Configurar un balanceador de carga utilizando Apache.
- Mejorar la resiliencia del sistema, garantizando alta disponibilidad y redundancia.

2.2 Componentes

El proyecto se apoya en tres máquinas virtuales previamente configuradas:

- **Máquina Virtual 1 – Base de Datos PostgreSQL y Balanceador de Carga**:

  Esta máquina aloja la base de datos PostgreSQL, ejecuta una instancia de la aplicación Django y actúa como balanceador de carga, distribuyendo las solicitudes entre las distintas instancias.

  - `Sistema Operativo`: Ubuntu
  - `Apache HTTP Server`: servidor web
  - *`Django`*: framework web en Python
  - *`PostgresSQL`*: sistema de gestión de bases de datos relacional
  - *`gunicorn`*: servidor WSGI que conecta *Django* con Apache

- **Máquina Virtual 2 – Instancia *Django***:

  Ejecuta una instancia de la aplicación *Django*, utilizando *gunicorn* como servidor WSGI.

  - `Sistema Operativo`: Ubuntu
  - *`Django`*
  - *`gunicorn`*

- **Máquina Virtual 3 – Instancia *Django***:

  Esta máquina es un clon funcional de la Máquina Virtual 2.

2.3 Flujo de datos

A continuación se describe el flujo de datos entre el cliente, el balanceador de carga, las instancias de *Django* y la base de datos PostgreSQL:

1. **Solicitud del cliente**: Un cliente (navegador web o API) realiza una petición HTTP a la aplicación *Django*.

2. **Recepción por el balanceador de carga**: Apache recibe la solicitud y la distribuye entre las instancias disponibles.
3. **Redirección de la solicitud**: Apache reenvía la petición a una instancia de *gunicorn*, por ejemplo, en la **VM2**.
4. **Procesamiento por *gunicorn***: *gunicorn* recibe la solicitud y la transmite a *Django* mediante la interfaz WSGI.
5. **Procesamiento por *Django***: *Django* ejecuta la lógica de negocio correspondiente.
6. **Consulta a la base de datos**: Si es necesario, *Django* realiza una consulta a *PostgresSQL* en la **VM1**.
7. **Generación de la respuesta**: *PostgresSQL* devuelve los datos, *Django* los procesa, *gunicorn* envía la respuesta a Apache, y Apache la entrega al cliente.

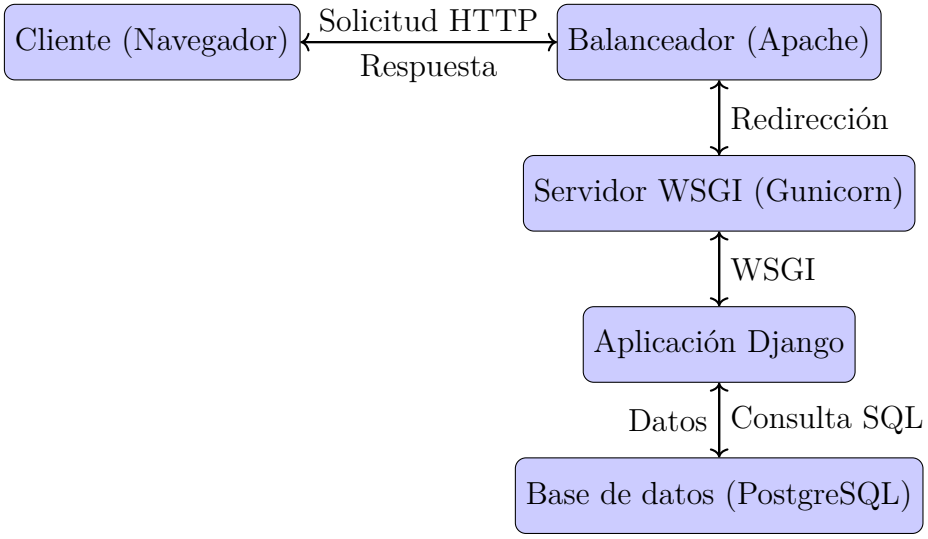

Figure 2: Flujo de interacción entre cliente, balanceador, aplicación y base de datos.

3 Instrucciones para la configuración y el despliegue

Partiendo de las tres máquinas virtuales con la aplicación **P1-base** instalada y en ejecución, esta sección describe el procedimiento para configurar el clúster en el entorno de los laboratorios.

Antes de comenzar, es necesario revertir los cambios realizados en la práctica P2 dentro del archivo `settings.py`, modificando la línea:

```
SESSION_ENGINE = django.contrib.sessions.backends.db
```

por:

```
SESSION_ENGINE = django.contrib.sessions.backends.cache
```

La opción `db` (frente a `cache`) almacena las variables de sesión en la base de datos (tabla `django_session`), lo que garantiza persistencia entre nodos del clúster, aunque con un mayor coste en rendimiento debido a las consultas constantes.

Por otro lado, la opción `cache` utiliza un caché local (por defecto, la memoria del sistema), lo que mejora la velocidad de acceso pero reduce la persistencia. En caso de reinicio del servidor de caché, las sesiones se pierden adicionalmente la información no se puede compartir entre nodos.

3.1 Configuración de Apache para el Balanceo de Carga

Para desplegar una aplicación *Django* en un entorno distribuido, cada nodo del clúster ejecuta una instancia de la aplicación. Para distribuir las peticiones de forma eficiente, se emplea Apache como balanceador de carga mediante el módulo `mod_proxy_balancer`.

3.1.1 Configuración del Proxy Balancer

Se debe crear o modificar el archivo de configuración de Apache para definir los servidores que componen el clúster y establecer las reglas de balanceo tal y como se muestra en el listado 1

Listado 1: Contenido del fichero `/etc/apache2/sites-available/000-default.conf`

```
ProxyRequests Off

# change ip_address by the worker nodes IP address
<Proxy balancer://SI2Cluster>
    BalancerMember http://ip_address:18000 route=Instance01
    BalancerMember http://ip_address:28000 route=Instance02
    BalancerMember http://ip_address:38000 route=Instance03

    ProxySet stickysession=ROUTEID
</Proxy>

Header add Set-Cookie "ROUTEID=.%{BALANCER_WORKER_ROUTE}e; path=/" env=BALANCER_ROUTE_CHANGED

<Location /P1base>
    Require all granted
    ProxyPass balancer://SI2Cluster
    ProxyPassReverse balancer://SI2Cluster
</Location>

<Location /balancer-manager>
    SetHandler balancer-manager
</Location>

RewriteEngine On
RewriteCond %{REQUEST_URI} !^/P1base
RewriteCond %{REQUEST_URI} !^/balancer-manager
RewriteRule ^(.*)$ /P1base$1 [R,L]
```

```

Header edit Location ^/(?!balancer-manager)([^/])
    ↪ /P1base/$1

# chage XX.YY.ZZ.II by the server IP number
ServerName XX.YY.ZZ.II
```

3.1.2 Explicación de las Directivas

- `ProxyRequests Off`: Desactiva el proxy directo, permitiendo únicamente el proxy inverso necesario para el balanceo de carga.
- `Header add Set-Cookie`: Añade una cookie `ROUTEID` que permite mantener la afinidad de sesión (sticky session) entre cliente e instancia.
- `BalancerMember`: Define los nodos del clúster, cada uno con su IP y puerto. La opción `route` identifica cada instancia para mantener la sesión persistente.
- `ProxySet stickysession=ROUTEID`: Configura el balanceador para usar la cookie `ROUTEID` como mecanismo de afinidad de sesión.
- `<Location /P1base>`: Establece las reglas de acceso y redirección para las peticiones dirigidas al clúster.
- `<Location /balancer-manager>`: Habilita el acceso a la interfaz de administración del balanceador de carga.
- `RewriteEngine` y `RewriteCond`: Gestionan las redirecciones para asegurar que las rutas se ajusten correctamente al contexto del balanceador.
- `Header edit Location`: Ajusta los encabezados de respuesta para evitar conflictos en las redirecciones.
- `ServerName`: Define el nombre del servidor o su dirección IP.

El servidor Apache cuando actúa como balanceador requiere los siguientes módulos extras:

```
sudo a2enmod proxy proxy_balancer proxy_http
    ↪ lbmethod_byrequests headers rewrite
```

y tras cualquier modificación de su configuración debe ser rebotado `sudo systemctl restart apache2.service` .

3.1.3 Verificación de la Configuración

Para comprobar que la configuración es correcta, se pueden ejecutar los siguientes comandos:

- `sudo apachectl configtest`: Verifica la sintaxis del archivo de configuración.
- `journalctl -xeu apache2.service`: Revisa los registros del servicio Apache.
- `sudo tail -n 50 /var/log/apache2/error.log`: Muestra los últimos errores registrados por Apache.

Una vez desplegado el balanceador, se puede acceder a la consola de administración mediante la URL `http://localhost:18080/balancer-manager`

La interfaz mostrada será similar a la figura 5.

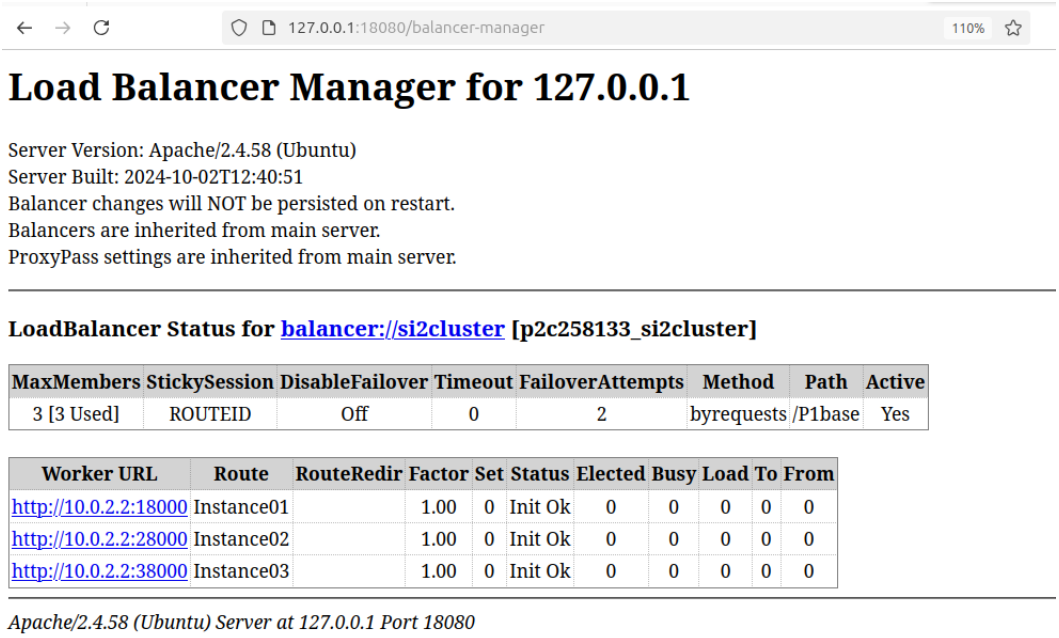

Figure 3: Interfaz de administración del balanceador de carga de Apache

Ejercicio 1

Incluya en la memoria de la practica pruebas de que se ha configurado y ejecutado el balanceador de carga de apache. Se debe incluir capturas de la ejecución `sudo systemctl status apache2` así como `sudo apachectl configtest` junto a capturas de la consola del balanceador.

Ejercicio 2

1. Mantenga únicamente en ejecución la instancia **VM1**. Para ello, apague las otras dos instancias mediante el comando:
   `sudo poweroff`
2. Reinicie el servicio de *Apache* en la máquina virtual activa utilizando:
   `sudo systemctl restart apache2.service`
3. Acceda al administrador del balanceador de carga en la siguiente URL:
   `http://ip_address:18080/balancer-manager`
   y capture una imagen del estado actual del balanceador.
4. Realice una petición a la aplicación web mediante:
   `curl http://ip_address:18080/P1base/visaApp/`
   y capture una imagen del estado actual del balanceador.
5. repita el punto anterior 2 veces más.
6. En total, debe obtener **cuatro capturas** del balanceador de carga. Incluya en la memoria las cuatro imágenes capturadas y redacte un comentario que responda a las siguientes cuestiones:
   - ¿En qué nodos se están ejecutando las peticiones?
   - ¿En qué momento detecta *Apache* que hay dos nodos apagados?
   - Usa las imágenes capturadas para apoyar tu respuesta.
7. Vuelve a rebotar apache (`sudo systemctl restart apache2.service`) y repite el ejercicio descrito pero esta vez en lugar de usar `curl` conéctate a `http://ip_address:18080/P1base/visaApp/` desde un navegador. ¿Los resultados son diferentes?¿Por qué?

4 Balanceo de carga

Para este ejercicio vamos a realizar unas modificaciones en la aplicación. Debemos añadir un nuevo campo en el modelo `Pago` de manera que también quede registrado la instancia desde la que se guardó.

```
instancia = models.CharField(max_length=24, default='None')
```

Será necesario regenerar la base de datos (makemigrations/migrate) y finalmente configuraremos las funciones encargadas de registrar el nuevo pago para que tengan en cuenta este nuevo valor. Para ello vamos a utilizar el valor de `ROUTEID` enviado en la petición.

```
pago_data['instancia'] = request.COOKIES.get('ROUTEID')
```

El siguiente paso será modificar la template usada en `get_pagos` para que muestre este nuevo atributo. Para ello añade el `pago.instancia` al bucle del template.

Con todo ello, cuando registremos un pago desde cada instancia se guardará (verificad la base de datos) y mostrará desde que instancia se realizó el pago en la pantalla "pago realizado con éxito":

Ejercicio 3

Pruebe a registrar un pago individualmente en cada instancia. Para ello, desde la consola de balanceo, fuerce el uso de una única instancia, compruebe que los pagos creados en una instancia se han registrado correctamente en la base de datos. En la memoria incluye imágenes de (1) la consola del Balanceador mostrando las dos instancias deshabilitadas, (2) la página del navegador comunicando que el pago se ha realizado con éxito y (3) finalmente la salida de una consulta en la base de datos que muestre que el pago se ha persistido en el sistema.

4.1 Afinidad de sesión

El objetivo de este ejercicio es probar cómo el balanceador de carga *Apache* maneja la afinidad de sesión utilizando la directiva ProxySet stickysession=ROUTEID.

Contexto: El balanceador de carga *Apache* está configurado para distribuir las solicitudes entre las instancias de una aplicación *Django*. La propiedad `stickysession` se configura con la cookie ROUTEID, que asegura que todas las solicitudes del mismo cliente se direccionen a la misma instancia del servidor, manteniendo la sesión activa incluso si el balanceador de carga distribuye las solicitudes a diferentes servidores.

Antes de comenzar las pruebas, asegúrate de eliminar todas las cookies almacenadas en tu navegador para garantizar que las pruebas se realicen de manera limpia y no haya interferencias con sesiones anteriores.

Se puede comprobar el valor de las cookie accediendo a las herramientas de desarrollador.

Filter Items

Name	Value	Domain	Path	Expires / Max-Age
ROUTEID	.Instance01	localhost	/	Session
sessionid	aywlhc8wqe6xcb0lob0h08e7fhgcdvht	localhost	/	Sat, 15 Mar 2025 20:00:16 GMT

Figure 4: Cookie afinidad de sesión

5 Pruebas balanceador

Ejercicio 4

Suprimiendo la afinidad de sesión, (en la consola del balanceador elimina la "sticky session") accede a la aplicación a través de la URL del balanceador. Realiza un pago suministrando información correcta, continua insertando pagos hasta que se produzca un fallo por afinidad de sesión.

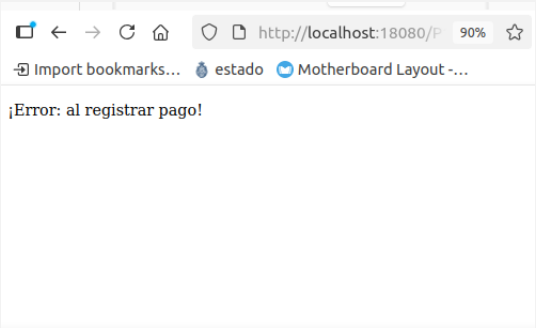

Figure 5: Error sesión

Explique en qué y cómo afecta el uso de ProxySet sticky session con respecto a errores.

Ejercicio 5

Verifica que el balanceador de carga maneja el failover correctamente, redirigiendo el tráfico a una nueva instancia del servidor si una de las instancias cae.

- Identifica la instancia que ha sido seleccionada menos veces: Utilizando el gestor de balanceo de carga (accediendo a la página `/balancer-manager)`, observa el número de "elecciones" para cada instancia. Esta métrica indicará cuántas solicitudes ha recibido cada servidor.
- Para la instancia del clúster con menos elecciones. Realza un `sudo poweroff` de la misma.
- Realiza peticiones nuevamente accediendo a la applicación web a través del balanceador. Accede a la página `/balancer-manager` para verificar que la instancia detenida ha sido marcada como "errónea" y que el balanceador está redirigiendo todas las peticiones a la instancia restante.
- Demuestra borrando cookies y accediendo varias veces que el balanceador redirige a la instancia que no está caída.

Incluye en la memoria una descripción del experimento realizado y los resultados obtenidos incluyendo capturas de pantalla de la consola del balanceador para apoyar tus conclusiones. Tus conclusiones deben citar las capturas de pantalla.

Ejercicio 6

Verifica que el balanceador de carga reactiva correctamente una instancia previamente detenida (failback).

- Inicia la instancia detenida. Como medida adicional, prueba a acceder directamente a la instancia desde el navegador sin pasar por el balanceador y compruebe que cargue la aplicación.
- Accede nuevamente a la página /balancer-manager del balanceador de carga y verifica que la instancia previamente caída se haya reactivado y esté ahora en funcionamiento. (Nota: puede ser necesario registrar un pago para que se actualice la información mostrada en esta página).
- Vuelve a realizar un pago completando los datos de la tarjeta como en ejercicios anteriores.
- Observa cómo el balanceador distribuye las solicitudes entre las instancias activas. Si el failback está funcionando correctamente, la carga debe distribuirse entre las instancias disponibles, incluyendo la recién reactivada.
- Incluye las evidencias del proceso de reactivación de la instancia, así como cualquier cambio visible en la consola del balanceador de carga.

Ejercicio 7

Verifica el comportamiento del balanceador y el proceso de failover en el transcurso de una sesión activa.

- Abre un navegador y comienza el proceso de pago rellenando los datos de la tarjeta de crédito en el formulario.
- Justo cuando la pantalla pida completar la información sobre el pago, observa la instancia que ha procesado la solicitud y detén esa instancia.
- Completar los datos del pago y enviar la petición.
- Observar la instancia del clúster que procesa el pago, y razona las causas por las que se rechaza la petición de registro del pago. Adjunta en la memoria las capturas de pantalla necesarias para justificar tu argumentación.

Ejercicio 8

Pruebas de carga con JMeter

- Abre el archivo de pruebas de jmeter generado en la práctica 3 para la aplicación P1base y modifica el ciclo de pruebas para realizar 1000 pagos en un solo hilo contra la IP del clúster y la nueva URL de la aplicación (Asegúrate de eliminar cualquier pago previo antes de iniciar el ciclo de pruebas para evitar interferencias. Puedes desactivar los thinktime para acelerar la prueba).
- Limpia los datos de apache de forma que ambas instancias empiecen con 0 datos.
- Después de ejecutar las pruebas, verifica cómo se distribuyen los pagos entre las instancias del clúster, observando los pagos registrados en la base de datos, ordenándolos por su marca de tiempo.
- Deduce qué algoritmo de reparto parece estar utilizando el balanceador de carga. Algunas preguntas clave a responder son:
  - ¿El balanceador distribuye las solicitudes de manera equitativa?
  - ¿Hay alguna instancia que reciba significativamente más o menos solicitudes que otras?
  - ¿El balanceador parece seguir un algoritmo de round-robin entre las instancias o un algoritmo de afinidad de sesión basado en la cookie ROUTEID, o ambos?
- Incluye las evidencias de los resultados obtenidos en JMeter y comenta los resultados.

Material a entregar

Se debe subir a *Moodle* un fichero en formato zip que contenga la memoria (en formato pdf) respondiendo a todos los ejercicios y cuestiones planteadas en esta guía junto con los ficheros de configuración y el código utilizada para implementar los diferentes proyectos. Se considerará que el código funciona si lo hace en los ordenadores del laboratorio de prácticas. El código entregado en *Moodle* será el evaluado, bajo ninguna condición se evaluará el código existente en otros repositorios como pueda ser *github*, las máquinas virtuales, etc..

6 Criterios de evaluación

Para aprobar es necesario haber implementado el balanceador de carga satisfaciendo los ejercicios 3, 5 y 6. En caso de no aprobar esos ejercicios, la nota máxima de la práctica será de un 4.9.

Una vez satisfecho este requiso la puntuación se estable según la tabla siguiente:

Ejercicio	**Puntuación**
1	0.5
2	0.5
3	1.5
4	1.0
5	1.5
6	1.5
7	1.5
8	1.0
Memoria	**1.0**